

Node.js Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This Node.js course is designed to take learners from JavaScript fundamentals to building production-ready backend and full-stack applications using Node.js.

The syllabus covers Node.js architecture, the event loop, modules, npm, file systems, streams, buffers, networking, REST APIs, Express, Fastify, authentication, authorization, databases, ORMs, testing, security, performance, caching, queues, WebSockets, microservices, Docker, CI/CD, observability, deployment, and project development.

The course is suitable for students, backend developers, full-stack developers, JavaScript/TypeScript developers, and working professionals who want to build scalable server-side applications using Node.js.

1. Prerequisites

Before starting Node.js, learners should have a working understanding of:

JavaScript Fundamentals

- Variables
- Data types
- Operators
- Conditional statements
- Loops
- Functions
- Arrays
- Objects
- Classes
- Scope
- Closures
- Error handling

Modern JavaScript

- let
- const
- Arrow functions
- Template literals
- Destructuring
- Spread operator
- Rest parameters
- Modules
- Promises
- Async/await

Basic Web Concepts

- Client-server architecture
- HTTP
- URLs
- JSON
- REST API basics

2. Introduction to Node.js

What is Node.js?

- JavaScript runtime
- V8 engine
- Server-side JavaScript
- Event-driven architecture

Why Node.js?

- Fast development
- JavaScript across frontend and backend
- Large ecosystem
- Real-time applications
- API development

Common Use Cases

- REST APIs
- Web applications
- Microservices
- Real-time applications
- CLI tools
- Serverless functions
- Streaming applications

Node.js vs Traditional Backend Platforms

- Node.js vs Java
- Node.js vs .NET
- Node.js vs Python
- Strengths and limitations

3. Node.js Architecture

V8 JavaScript Engine

Node.js Runtime

libuv

Event-Driven Architecture

Non-Blocking I/O

Single-Threaded JavaScript Execution

Background Thread Pool

Concurrency vs Parallelism

4. Event Loop

Event Loop Fundamentals

Event Loop Phases

- Timers
- Pending callbacks
- Poll
- Check
- Close callbacks

Microtasks

- Promise callbacks
- queueMicrotask

`process.nextTick`

`setTimeout`

`setImmediate`

Event Loop Ordering**Avoiding Event Loop Blocking****5. Installing and Setting Up Node.js****Node.js Installation****Node Version Managers**

- nvm concepts
- Version switching

Checking Versions

- `node -v`
- `npm -v`

Running JavaScript Files**Node REPL****VS Code Setup****6. npm Fundamentals****What is npm?**

`package.json`

Initializing a Project

- `npm init`
- `npm init -y`

Installing Packages

- Production dependencies
- Development dependencies

Removing Packages**Updating Packages****Semantic Versioning****npm Scripts****7. Package Management**

`package-lock.json`

Dependency Trees

Local vs Global Packages

npm Cache

npm audit

Alternative Package Managers

- pnpm
- yarn

Workspaces Concepts

8. Modules

CommonJS

- require
- module.exports

ES Modules

- import
- export

ESM vs CommonJS

File Extensions

- .js
- .mjs
- .cjs

type in package.json

Module Interoperability

9. Core Node.js Modules

fs

path

os

url

events

http

https

crypto

stream

buffer

util

10. File System

fs Module

Reading Files

Writing Files

Appending Files

Renaming Files

Deleting Files

Creating Directories

Reading Directories

File Metadata

11. Asynchronous File Operations

Callback-Based APIs

Promise-Based File System API

Async/await with File Operations

Error Handling

Synchronous vs Asynchronous File Operations

Performance Considerations

12. Path Module

Working with Paths

`path.join`

`path.resolve`

`path.basename`

`path.dirname`

`path.extname`

Cross-Platform Paths

13. URL Module

URL Parsing

URL Objects

Search Parameters

Query Strings

Pathnames

Working with Web URLs

14. Buffer

What is a Buffer?

Binary Data**Creating Buffers****Reading Buffers****Writing Buffers****Buffer Encoding**

- UTF-8
- Base64
- Hex

Buffer Use Cases**15. Streams****Stream Fundamentals****Types of Streams**

- Readable
- Writable
- Duplex
- Transform

Reading Streams**Writing Streams****Piping Streams****Backpressure****Stream Error Handling****16. Advanced Streams****Transform Streams****Pipeline API****Async Iteration over Streams****Streaming Large Files****Compression Streams****Memory-Efficient Processing****17. Events****EventEmitter****Creating Events****Listening to Events****Passing Event Data****Removing Listeners****One-Time Listeners**

Error Events**Event-Driven Design****18. Process Object**`process.argv`**Environment Variables****Process ID****Current Working Directory****Standard Input****Standard Output****Standard Error****Process Exit Codes****19. Environment Variables**`process.env`**.env Files****Environment Configuration**

- Development
- Testing
- Staging
- Production

Secret Management Concepts**Configuration Validation****20. Child Processes**`child_process``exec``execFile``spawn``fork`**Use Cases****Security Considerations****21. Worker Threads****Worker Thread Fundamentals****CPU-Bound Tasks****Creating Workers**

Passing Data

Shared Memory Concepts

Worker Pools

When to Use Worker Threads

22. Cluster and Multi-Core Concepts

Node.js and CPU Cores

Process Clustering Concepts

Load Distribution

Process Managers

Horizontal Scaling

Modern Container-Oriented Scaling

23. HTTP Fundamentals

HTTP Request

HTTP Response

HTTP Methods

- GET
- POST
- PUT
- PATCH
- DELETE

HTTP Headers

Status Codes

Request Body

Query Parameters

Path Parameters

24. Creating an HTTP Server

http Module

Creating a Server

Request Object

Response Object

Sending Responses

Handling Routes Manually

JSON Responses

25. HTTPS

HTTPS Fundamentals

TLS Concepts

Certificates

Secure Server Setup Concepts

Production Reverse Proxy

26. REST API Fundamentals

What is REST?

Resources

Endpoints

HTTP Methods

Status Codes

JSON

Statelessness

Idempotency

REST Naming Conventions

27. REST API Design

Resource-Oriented URLs

CRUD Operations

Pagination

Filtering

Sorting

Searching

Versioning

Error Response Standards

28. Express.js Introduction

What is Express?

Installing Express

Creating an Express Server

Routes

Middleware

Request and Response Objects

JSON Parsing

29. Express Routing

GET Routes

POST Routes

PUT Routes

PATCH Routes

DELETE Routes

Route Parameters

Query Parameters

Router Objects

30. Express Middleware

Middleware Fundamentals

Application Middleware

Router Middleware

Built-In Middleware

Custom Middleware

Third-Party Middleware

Middleware Ordering

31. Error Handling in Express

Synchronous Errors

Asynchronous Errors

Error Middleware

Centralized Error Handling

Custom Error Classes

HTTP Error Responses

Production Error Handling

32. Fastify Introduction

Why Fastify?

Fastify Server Setup

Routes

Plugins

Hooks

Schemas

Validation

Serialization

Express vs Fastify

33. API Validation

Why Validation Matters

Request Body Validation

Query Validation

Route Parameter Validation

Schema Libraries

- Zod
- Joi concepts
- Ajv concepts

Validation Error Responses

34. API Serialization

Response Models

DTOs

Field Selection

Preventing Sensitive Data Leakage

Consistent Response Shapes

35. API Error Design

Error Codes

HTTP Status Codes

Validation Errors

Authentication Errors

Authorization Errors

Resource Errors

Internal Errors

Correlation IDs

36. Logging

Application Logging

Structured Logging

Log Levels

- Debug
- Info
- Warn
- Error

Request Logging

Correlation IDs

Logging Libraries

- Pino
- Winston

37. Database Fundamentals

Relational vs NoSQL Databases

SQL Databases

- PostgreSQL
- MySQL
- SQL Server

NoSQL Databases

- MongoDB
- Redis

Choosing a Database

38. PostgreSQL with Node.js

Connecting to PostgreSQL

Connection Pools

Queries

Parameterized Queries

Transactions

Error Handling

SQL Injection Prevention

39. MySQL with Node.js

Connecting to MySQL

Querying

Prepared Statements

Transactions

Connection Pools

40. MongoDB with Node.js

MongoDB Fundamentals

Documents

Collections

CRUD Operations

Query Operators

Indexes

Aggregation Basics

41. Mongoose

Mongoose Setup

Schemas

Models

CRUD Operations

Validation

Middleware/Hooks

Population

Indexes

42. ORM Fundamentals

What is an ORM?

ORM vs Raw SQL

Benefits

Tradeoffs

Common Node.js ORMs

- Prisma
- TypeORM concepts
- Sequelize concepts

43. Prisma

Prisma Setup

Schema

Models

Prisma Client

Migrations

CRUD Operations

Relations

Transactions

Seeding

44. Drizzle ORM Concepts

Type-Safe SQL

Schema Definitions

Queries

Relations

Migrations

Prisma vs Drizzle Concepts

45. Database Design

Primary Keys

Foreign Keys

Relationships

- One-to-One
- One-to-Many
- Many-to-Many

Normalization

Indexes

Constraints

Auditing Fields

46. Database Transactions

ACID Concepts

Transactions

Commit

Rollback

Isolation Concepts

Transaction Use Cases

47. Database Performance

Indexing

Query Optimization

N+1 Problem

Pagination

Connection Pooling

Caching

Database Monitoring

48. Authentication Fundamentals

Authentication vs Authorization

User Registration

Login

Logout

Password Storage

Sessions

Tokens

49. Password Security

Password Hashing

Salt

bcrypt

Argon2 Concepts

Password Policies

Password Reset

Never Store Plain-Text Passwords

50. JWT Authentication

JWT Structure

Header

Payload

Signature

Access Tokens

Refresh Tokens

Token Expiration

Token Verification

51. Session-Based Authentication

Sessions

Session IDs

Cookies

Server-Side Session Storage

Redis Sessions

Session Expiration

Session Revocation

52. Cookies

Cookie Fundamentals

HTTP-Only Cookies

Secure Cookies

SameSite

Cookie Expiration

Signed Cookies

Security Considerations

53. Authorization

Role-Based Access Control

Permission-Based Access Control

Middleware Authorization

Resource Ownership

Admin Routes

Backend Enforcement

54. OAuth 2.0 and OpenID Connect

OAuth Fundamentals

Authorization Code Flow Concepts

OpenID Connect

Access Tokens

ID Tokens

Common Providers

- Google
- GitHub
- Microsoft Entra ID

55. API Security

Input Validation

Authentication

Authorization

Rate Limiting

Secure Headers

CORS

HTTPS

Dependency Security

56. CORS

Same-Origin Policy

Cross-Origin Requests

Allowed Origins

Methods

Headers

Credentials

Secure Production Configuration

57. Security Headers

Helmet Concepts

Content Security Policy

HSTS

X-Content-Type-Options

Frame Protection

Referrer Policy

58. Rate Limiting

Why Rate Limiting?

IP-Based Limiting

User-Based Limiting

Distributed Rate Limiting

Redis-Based Limiting

Brute-Force Protection

59. Common Web Security Risks

SQL Injection

NoSQL Injection

XSS

CSRF

Command Injection

Path Traversal

SSRF Concepts

Insecure Deserialization Concepts

60. File Uploads

Multipart Forms

Upload Middleware

File Validation

MIME Types

File Size Limits

File Naming

Security Risks

61. Object Storage

Cloud File Storage Concepts

Amazon S3

Azure Blob Storage

Google Cloud Storage

Presigned URLs

Direct-to-Cloud Uploads

62. Sending Email

Transactional Email

SMTP Concepts

Email Providers

Email Templates

Verification Emails

Password Reset Emails

Error Handling

63. Background Jobs

Why Background Jobs?

Job Queues

Producers

Consumers

Retry Policies

Delayed Jobs

Scheduled Jobs

64. Redis

Redis Fundamentals

Key-Value Storage

Caching

Sessions

Counters

Rate Limiting

Pub/Sub Concepts

65. BullMQ Concepts

Job Queues with Redis

Workers

Jobs

Retries

Backoff

Delayed Jobs

Repeatable Jobs

Dead-Letter Concepts

66. Caching

Application Caching

Database Query Caching

HTTP Caching

Redis Caching

Cache-Aside Pattern

TTL

Cache Invalidation

67. WebSockets

WebSocket Fundamentals

Persistent Connections

Bi-Directional Communication

Server Setup

Client Connections

Rooms/Channels Concepts

Real-Time Use Cases

68. Socket.IO

Socket.IO Fundamentals

Events

Rooms

Namespaces

Reconnection

Broadcasting

Authentication

69. Server-Sent Events

SSE Fundamentals

One-Way Streaming

Event Streams

SSE vs WebSockets

Use Cases

70. Real-Time Application Design

Chat Applications

Notifications

Live Dashboards

Collaboration

Presence

Scaling Real-Time Systems

71. TypeScript with Node.js

Why TypeScript for Node.js?

Project Setup

TypeScript Compiler

ESM Configuration

Node.js Type Definitions

Strict Mode

72. Type-Safe Backend Development

Request Types

Response Types

DTOs

Database Types

Service Types

Error Types

API Contracts

73. Clean Architecture

Presentation Layer

Application Layer

Domain Layer

Infrastructure Layer

Dependency Direction

Testability

74. Layered Backend Architecture

Routes

Controllers

Services

Repositories

Models

Validators

Middleware

Utilities

75. Controller-Service-Repository Pattern

Controllers

- HTTP concerns

Services

- Business logic

Repositories

- Data access

Benefits

- Separation of concerns
- Testing
- Maintainability

76. Dependency Injection

Dependency Injection Concepts

Constructor Injection

Service Containers

Testing Benefits

NestJS DI Concepts

77. NestJS Introduction

What is NestJS?

Modules

Controllers

Providers

Services

Dependency Injection

Decorators

Enterprise Backend Development

78. NestJS Features

Pipes

Guards

Interceptors

Exception Filters

Middleware

Validation

Modules

Testing

79. GraphQL with Node.js

GraphQL Fundamentals

Schema

Queries

Mutations

Resolvers

Types

REST vs GraphQL

80. GraphQL Servers

Apollo Server Concepts

GraphQL Yoga Concepts

Context

Authentication

Authorization

Error Handling

Data Loaders Concepts

81. API Documentation

OpenAPI

Swagger

Documenting Endpoints

Request Schemas

Response Schemas

Authentication Documentation

Interactive API Docs

82. API Versioning

URL Versioning

Header Versioning

Backward Compatibility

Deprecation

Migration Strategy

83. Pagination

Offset Pagination

Cursor Pagination

Limit and Offset

Sorting

Stable Ordering

Large Dataset Considerations

84. Search and Filtering

Query Parameters

Filtering

Sorting

Full-Text Search Concepts

Search Engines

- Elasticsearch concepts
- OpenSearch concepts

85. Testing Fundamentals

Unit Testing

Integration Testing

End-to-End Testing

Test Pyramid

Test Coverage

Test Isolation

86. Vitest and Jest

Test Suites

Test Cases

Assertions

Mocks

Spies

Async Testing

Setup and Teardown

87. API Testing

Supertest Concepts

Testing Routes

Testing Status Codes

Testing Response Bodies

Testing Headers

Authentication Tests

Error Cases

88. Database Testing

Test Databases

Transactions in Tests

Seeding

Cleanup

Repository Tests

Integration Testing

89. Mocking External Services

Mock HTTP APIs

Mock Email

Mock Storage

Mock Payment Providers

Test Doubles

Contract Testing Concepts

90. End-to-End Testing

E2E Test Environment

Full User Flows

Authentication

Database State

External Dependency Strategies

91. Debugging

Node Inspector

VS Code Debugging

Breakpoints

Call Stack

Variables

Async Debugging

Logging

92. Error Handling Strategy

Operational Errors

Programmer Errors

Centralized Error Handling

Safe Error Responses

Error Codes

Logging Errors

Process-Level Error Handling

93. Unhandled Errors

Unhandled Promise Rejections

Uncaught Exceptions

Graceful Shutdown

Process Recovery

Avoiding Silent Failures

94. Graceful Shutdown

SIGTERM

SIGINT

Closing HTTP Servers

Closing Database Connections

Finishing Jobs

Container Shutdown

95. Performance Fundamentals

CPU vs I/O Workloads

Event Loop Lag

Memory Usage

Garbage Collection Concepts

Throughput

Latency

96. Performance Profiling

CPU Profiling

Heap Snapshots

Memory Leak Detection

Flame Graph Concepts

Node.js Performance Hooks

Benchmarking

97. Avoiding Event Loop Blocking

Large Loops

CPU-Intensive Computation

Synchronous File APIs

Large JSON Operations

Worker Threads

Background Services

98. Memory Management

Heap

Stack

Garbage Collection

Memory Leaks

Event Listener Leaks

Cache Leaks

Global References

99. Scaling Node.js Applications

Vertical Scaling

Horizontal Scaling

Multiple Instances

Load Balancers

Stateless Services

Shared Cache

Shared Session Store

100. Load Balancing

Reverse Proxy

Nginx Concepts

Cloud Load Balancers

Health Checks

Round Robin

Sticky Sessions Concepts

101. Microservices Fundamentals

Monolith vs Microservices

Service Boundaries

Independent Deployment

Database per Service Concepts

Service Communication

Distributed System Tradeoffs

102. Inter-Service Communication

REST

gRPC Concepts

Messaging

Event-Driven Architecture

Synchronous vs Asynchronous Communication

103. Message Brokers

RabbitMQ Concepts

Apache Kafka Concepts

Producers

Consumers

Topics

Queues

Consumer Groups

104. Event-Driven Architecture

Events

Commands

Producers

Consumers

Eventual Consistency

Idempotency

Event Schemas

105. Idempotency

Idempotent APIs

Idempotency Keys

Payment APIs

Retry Safety

Duplicate Message Handling

106. Distributed System Patterns

Retry

Timeout

Circuit Breaker

Bulkhead Concepts

Backoff

Dead Letter Queues

Saga Concepts

107. Serverless Node.js

Serverless Fundamentals

Functions as a Service

Cold Starts

Stateless Execution

AWS Lambda Concepts

Azure Functions Concepts

Google Cloud Functions Concepts

108. Docker Fundamentals

Containers

Images

Dockerfile

Build

Run

Port Mapping

Environment Variables

109. Dockerizing Node.js Applications

Base Images

Multi-Stage Builds

Dependency Installation

Production Dependencies

Non-Root Users

Health Checks

Image Optimization

110. Docker Compose

Multi-Container Applications

Node.js Service

PostgreSQL

Redis

Environment Files

Networks

Volumes

111. CI/CD

Continuous Integration

Automated Linting

Automated Testing

Build

Security Scans

Continuous Deployment

112. GitHub Actions

Workflow Files

Triggers

Jobs

Steps

Secrets

Node.js CI Pipeline

Deployment Workflows

113. Cloud Deployment

AWS

- EC2 concepts
- ECS concepts
- Lambda concepts

Azure

- App Service
- Container Apps
- Functions

Google Cloud

- Cloud Run
- Functions

Platform-as-a-Service

- Render
- Railway concepts

114. Production Process Management

Process Managers

PM2 Concepts

Restart Policies

Logs

Cluster Mode Concepts

Container Platforms vs PM2

115. Reverse Proxy

Nginx

Request Forwarding

SSL Termination

Compression

Static Files

Load Balancing

116. HTTPS in Production

TLS Certificates

Let's Encrypt Concepts

Certificate Renewal

Reverse Proxy TLS

Cloud-Managed Certificates

117. Observability

Logs

Metrics

Traces

Correlation IDs

Dashboards

Alerts

Service-Level Indicators Concepts

118. Metrics

Request Count

Error Rate

Response Time

CPU

Memory

Event Loop Lag

Database Pool Metrics

119. Distributed Tracing

Trace IDs

Span IDs

Request Context

OpenTelemetry Concepts

Cross-Service Tracing

Performance Analysis

120. Health Checks

Liveness

Readiness

Startup Checks

Database Connectivity

Dependency Health

Kubernetes-Friendly Endpoints

121. Kubernetes Concepts for Node.js Developers

Pods

Deployments

Services

ConfigMaps

Secrets

Health Probes

Horizontal Pod Autoscaling Concepts

122. Configuration Management

Environment Variables

Configuration Modules

Validation

Secret Stores

Development/Staging/Production Differences

Twelve-Factor App Concepts

123. Secrets Management

Why Secrets Must Not Be Hardcoded

Environment Secrets

Cloud Secret Stores

Vault Concepts

Rotation

Access Control

124. API Gateway Concepts

Gateway Responsibilities

Routing

Authentication

Rate Limiting

Logging

Request Transformation

Aggregation

125. Backend for Frontend

BFF Pattern

Web BFF

Mobile BFF

Aggregating Downstream Services

Reducing Frontend Complexity

126. Design Patterns in Node.js

Singleton

Factory

Strategy

Adapter

Repository

Observer

Dependency Injection

Middleware Pattern

127. SOLID Principles

Single Responsibility Principle

Open/Closed Principle

Liskov Substitution Principle

Interface Segregation Principle

Dependency Inversion Principle

Applying SOLID in Node.js

128. Clean Code

Naming

Small Functions

Separation of Concerns

Avoiding Duplication

Error Handling

Comments

Code Reviews

129. ESLint and Formatting

ESLint

Flat Configuration Concepts

TypeScript ESLint

Prettier

Formatting Rules

Pre-Commit Checks

130. Git and GitHub

Repository Management

Branches

Commits

Pull Requests

Code Reviews

Merge Conflicts

Release Tags

131. Monorepos

Monorepo Concepts

npm Workspaces

pnpm Workspaces

Turborepo Concepts

Nx Concepts

Shared Packages

132. Building CLI Applications

Command-Line Arguments

Interactive Prompts

File Operations

CLI Framework Concepts

Packaging CLI Tools

Publishing

133. Building npm Packages

Package Structure

Public API

Semantic Versioning

npm Publishing

ESM/CJS Support

TypeScript Declaration Files

134. OpenAPI-First Development

API Contract

OpenAPI Schema

Code Generation

Validation

Documentation

Frontend Client Generation

135. Contract Testing

Consumer Contracts

Provider Contracts

API Compatibility

Schema Validation

Breaking Change Detection

136. AI Integration with Node.js

Calling LLM APIs

Server-Side API Keys

Streaming Responses

Tool Calling Concepts

Structured Outputs

Error Handling

137. RAG Application Concepts

Retrieval-Augmented Generation

Embeddings

Vector Databases

Document Retrieval

Prompt Construction

Node.js Backend Role

138. AI Agent Backend Concepts

Agents

Tools

Workflow Orchestration

State

Guardrails

Human-in-the-Loop

Observability

139. AI-Assisted Node.js Development

Using Coding Assistants

- API scaffolding
- Test generation
- Refactoring
- Documentation
- Debugging

Prompting for Backend Code

- Runtime version
- Framework
- Database
- Security requirements

AI Code Review

- Validation
- Authentication

- Async errors
- SQL injection
- Resource leaks

Responsible Usage

- Verify generated code
- Protect secrets
- Review dependencies

140. Mini Project 1 – File Management CLI

Features

- Create files
- Read files
- Rename files
- Delete files
- List directories

Concepts

- File system
- Path
- CLI arguments
- Error handling

141. Mini Project 2 – REST API

Features

- CRUD
- Validation
- Error handling
- Logging

Concepts

- Express/Fastify
- Routing
- Middleware
- REST design

142. Mini Project 3 – Authentication API

Features

- Registration
- Login
- Logout
- JWT/session authentication
- Role-based authorization

Concepts

- Security
- Password hashing
- Tokens
- Middleware

143. Mini Project 4 – Employee Management System

Features

- Employees
- Departments
- CRUD
- Search
- Pagination

Concepts

- PostgreSQL/MongoDB
- ORM
- API design
- Validation

144. Mini Project 5 – Real-Time Chat Backend**Features**

- User connections
- Rooms
- Messages
- Presence
- Notifications

Concepts

- WebSockets
- Socket.IO
- Authentication
- Real-time systems

145. Mini Project 6 – Background Job Processor**Features**

- Job queue
- Worker
- Retry
- Delayed jobs
- Status tracking

Concepts

- Redis
- BullMQ
- Background processing

146. Mini Project 7 – API Gateway / BFF**Features**

- Multiple downstream APIs
- Authentication
- Aggregated responses
- Rate limiting
- Logging

Concepts

- Service composition
- API gateway

- Error handling
- Resilience

147. Major Capstone Project – E-Commerce Backend

Features

- User management
- Authentication
- Products
- Categories
- Cart
- Orders
- Payments concepts
- Inventory
- Admin APIs

Technical Requirements

- Node.js
- TypeScript
- Express/Fastify/NestJS
- PostgreSQL/MongoDB
- Redis
- Authentication
- Authorization
- Validation
- Testing
- Docker
- Deployment

148. Major Capstone Project – Learning Management Backend

Features

- Users
- Students
- Instructors
- Courses
- Lessons
- Enrollment
- Assessments
- Progress
- Admin APIs

Advanced Concepts

- Role-based access
- File storage
- Background jobs
- Notifications
- Reporting

149. Major Capstone Project – SaaS Project Management Backend

Features

- Organizations

- Teams
- Projects
- Tasks
- Comments
- Notifications
- Audit logs

Advanced Concepts

- Multi-tenancy
- Authorization
- Real-time updates
- Queues
- Caching
- Observability

150. Capstone Project Requirements

The final project should demonstrate:

- Node.js fundamentals
- TypeScript
- REST API design
- Express/Fastify/NestJS
- Modular architecture
- Validation
- Database integration
- Authentication
- Authorization
- Error handling
- Logging
- Caching
- Background jobs
- Testing
- Security
- Docker
- CI/CD
- Deployment
- API documentation
- Git/GitHub
- README documentation

151. Interview Preparation – Node.js Fundamentals

Important Topics

- Node.js architecture
- V8
- libuv
- Event loop
- Non-blocking I/O
- EventEmitter
- Buffers
- Streams
- CommonJS vs ESM
- npm

152. Interview Preparation – Backend

Important Topics

- REST API design
- Middleware
- Authentication
- JWT
- Sessions
- Authorization
- Validation
- Error handling
- Database transactions
- Indexing
- Caching
- Queues
- WebSockets

153. Interview Preparation – Advanced

Advanced Topics

- Event loop phases
- Worker threads
- Memory leaks
- Event loop blocking
- Scaling
- Microservices
- Idempotency
- Distributed transactions
- Circuit breakers
- Observability
- Graceful shutdown

154. Practical Interview Exercises

Candidates should be able to:

- Create an HTTP server
- Build a REST API
- Write middleware
- Implement centralized error handling
- Validate requests
- Implement authentication
- Protect routes
- Connect a database
- Write a transaction
- Add Redis caching
- Implement a queue worker
- Write API tests
- Dockerize an application

155. Coding Exercises

Students should practice:

- File reader

- CLI application
- HTTP server
- URL parser
- Event emitter
- Stream processor
- REST CRUD API
- Pagination API
- Authentication middleware
- Rate limiter
- Cache wrapper
- Queue processor
- WebSocket server
- Graceful shutdown handler
- Generic repository

156. Recommended Modern Node.js Stack

A modern Node.js development stack may include:

- Node.js LTS
- TypeScript
- npm or pnpm
- Express or Fastify
- NestJS for enterprise applications
- PostgreSQL
- MongoDB where appropriate
- Prisma or Drizzle
- Redis
- BullMQ
- Zod
- Pino
- Vitest or Jest
- Supertest
- OpenAPI/Swagger
- Docker
- Git
- GitHub
- CI/CD
- OpenTelemetry

157. Suggested Course Sequence

Phase 1 – JavaScript and Node.js Foundation

- JavaScript refresher
- Modern JavaScript
- Node.js introduction
- Architecture
- Event loop
- npm
- Modules
- Core modules

Phase 2 – Core Backend Development

- File system

- Streams
- Events
- HTTP
- REST APIs
- Express/Fastify
- Middleware
- Validation
- Error handling

Phase 3 – Data Layer

- SQL fundamentals
- PostgreSQL/MySQL
- MongoDB
- ORM
- Prisma/Drizzle
- Transactions
- Indexing

Phase 4 – Security and Authentication

- Authentication
- Password security
- JWT
- Sessions
- Authorization
- CORS
- Rate limiting
- Security headers

Phase 5 – Advanced Backend

- Redis
- Caching
- Background jobs
- WebSockets
- File uploads
- Email
- API documentation

Phase 6 – Architecture and Scale

- TypeScript
- Clean architecture
- Microservices
- Messaging
- Resilience patterns
- Performance
- Observability

Phase 7 – Production Engineering

- Testing
- Docker
- CI/CD
- Cloud deployment
- Kubernetes concepts
- Security hardening

Phase 8 – Career Preparation

- Mini projects
- Capstone project
- GitHub portfolio
- Interview preparation
- Mock interviews

158. Learning Outcomes

After completing this Node.js course, learners should be able to:

- Understand Node.js architecture and the event loop
- Build server-side applications using JavaScript and TypeScript
- Work with modules, files, buffers, and streams
- Build production-ready REST APIs
- Use Express, Fastify, or NestJS
- Design request validation and error handling
- Integrate SQL and NoSQL databases
- Use ORMs effectively
- Implement secure authentication and authorization
- Apply API security best practices
- Use Redis for caching and sessions
- Build background job systems
- Build real-time applications with WebSockets
- Write unit, integration, and API tests
- Optimize Node.js performance
- Build scalable and resilient services
- Use Docker and CI/CD
- Deploy Node.js applications to cloud platforms
- Implement logging, metrics, and tracing
- Build portfolio-ready backend projects
- Prepare for Node.js backend interviews

159. Suggested Course Deliverables

By the end of the course, each learner should ideally have:

- Node.js coding assignments
- Event loop exercises
- File system exercises
- REST API assignments
- Database integration assignments
- Authentication assignments
- Redis and caching exercises
- Queue-processing exercises
- Real-time application exercises
- 5–7 mini projects
- At least one major capstone project
- GitHub repositories
- API documentation
- Automated tests
- Dockerized applications
- CI/CD workflow
- Deployed backend application
- Interview preparation
- Mock interview experience

- AI-assisted development practice
- A backend development portfolio

160. Optional Advanced Modules

Advanced Node.js Internals

- V8 internals
- libuv internals
- Native addons concepts
- N-API concepts

Advanced Performance

- Profiling
- Flame graphs
- Heap analysis
- Worker pools
- High-throughput architecture

Advanced Distributed Systems

- Event sourcing
- CQRS
- Saga pattern
- Outbox pattern
- Distributed locks

Advanced Cloud-Native Development

- Kubernetes
- Service mesh concepts
- Autoscaling
- Managed databases
- Managed messaging

Advanced API Design

- GraphQL
- gRPC
- AsyncAPI
- Event APIs

161. Career Roles After Node.js Training

This course can help learners prepare for roles such as:

- Node.js Developer
- Backend Developer
- JavaScript Backend Developer
- TypeScript Backend Developer
- API Developer
- Full-Stack Developer
- Backend Software Engineer
- Microservices Developer
- Cloud Backend Developer
- Serverless Developer

Conclusion

Node.js is one of the most widely used platforms for building modern backend, API, real-time, and cloud-native applications using JavaScript and TypeScript.

A production-ready Node.js developer should understand much more than Express routing. They should know the event loop, asynchronous programming, streams, database design, authentication, security, caching, queues, testing, performance, observability, microservices, containers, deployment, and scalable backend architecture.

This syllabus provides a structured path from JavaScript fundamentals to professional Node.js backend development and interview preparation.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.